To estimate the robot pose, we use multiple ArUco markers. The markers are detected using OpenCV's ArUco module. We then solve the perspective-N-point problem of all detected marker corners jointly, using the SQPnP algorithm. As we define the marker corners' 3D positions in the world frame, this yields the world-to-camera transform ${}^{\mathrm{C}}\boldsymbol{H}_{\mathrm{W}}$. Inverting this yields the camera pose in the world frame ${}^{\mathrm{W}}\boldsymbol{H}_{\mathrm{C}}$. Using the known transformation from the camera to the robot base ${}^{\mathrm{B}}\boldsymbol{H}_{\mathrm{C}}$, we obtain the required robot pose in the world frame

$$ {}^{\mathrm{W}}\boldsymbol{H}_{\mathrm{B}} = ({}^{\mathrm{C}}\boldsymbol{H}_{\mathrm{W}})^{-1}({}^{\mathrm{B}}\boldsymbol{H}_{\mathrm{C}})^{-1}. $$

In order to integrate the ArUco pose estimation into a localization pipeline, we need to estimate the camera pose covariance. We first compute the image Jacobian $\boldsymbol{J}$ of the marker image coordinates $\boldsymbol{m} = [u, v]^\top$ with respect to the camera pose

$$ \boldsymbol{J} = \frac{\partial \boldsymbol{m}}{\partial \boldsymbol{\xi}} = \frac{\partial \boldsymbol{m}}{\partial {}^{\mathrm{C}}\boldsymbol{p}} \frac{\partial {}^{\mathrm{C}}\boldsymbol{p}}{\partial \boldsymbol{\xi}}, $$

where ${}^{\mathrm{C}}\boldsymbol{p}$ are the marker points in the camera frame and $\boldsymbol{\xi}$ are small perturbations in the camera pose. Consequently,

$$ \dot{\boldsymbol{m}} = \boldsymbol{J} \begin{pmatrix} {}^{\mathrm{C}}\boldsymbol{\nu}_{\mathrm{C}} \\ {}^{\mathrm{C}}\boldsymbol{\omega}_{\mathrm{C}} \end{pmatrix}, $$

where ${}^{\mathrm{C}}\boldsymbol{\nu}_{\mathrm{C}}$ and ${}^{\mathrm{C}}\boldsymbol{\omega}_{\mathrm{C}}$ are the translational and rotational velocities of the camera in the camera frame. The outer product of the Jacobian with itself is used as an approximation (Gauss-Newton) of the Hessian, and the inverse of this is used as an approximation (Gaussian model) of the camera pose covariance in the camera frame

$$ {}^{\mathrm{C}}\boldsymbol{\Sigma}_{\mathrm{C}} \approx (\boldsymbol{J}^\top \boldsymbol{J})^{-1}\sigma^2, $$

where σ is the root-mean-squared reprojection error of the marker points.

As we require the pose covariance in the world frame, ${}^{\mathrm{C}}\boldsymbol{\Sigma}_{\mathrm{C}}$ is then rotated into the world frame using

$$ \boldsymbol{A}_{\mathrm{CW}} = \begin{pmatrix} {}^{\mathrm{W}}\boldsymbol{R}_{\mathrm{C}} & \boldsymbol{0} \\ \boldsymbol{0} & {}^{\mathrm{W}}\boldsymbol{R}_{\mathrm{C}} \end{pmatrix} \tag{1} $$

$$ {}^{\mathrm{W}}\boldsymbol{\Sigma}_{\mathrm{C}} = \boldsymbol{A}_{\mathrm{CW}}\, {}^{\mathrm{C}}\boldsymbol{\Sigma}_{C} \boldsymbol{A}_{\mathrm{CW}}^\top. \tag{2} $$

Finally, we need to obtain the robot pose covariance in the world frame ${}^{\mathrm{W}}\boldsymbol{\Sigma}_{\mathrm{B}}$. As both ${}^{\mathrm{W}}\boldsymbol{\Sigma}_{\mathrm{C}}$ and ${}^{\mathrm{W}}\boldsymbol{\Sigma}_{\mathrm{B}}$ are given in the world frame, the angular components do not change. However, the base position covariance suffers the lever-arm effect of the camera's angular uncertainty. The final covariance is given by

$$ {}^{\mathrm{W}}\boldsymbol{d} = {}^{\mathrm{W}}\boldsymbol{R}_{\mathrm{C}}\, {}^{\mathrm{C}}\boldsymbol{o}_{\mathrm{B}} \tag{3} $$

$$ \boldsymbol{A}_{\mathrm{CB}} = \begin{pmatrix} \boldsymbol{I} & -[{}^{\mathrm{W}}\boldsymbol{d}]_\times \\ \boldsymbol{0} & \boldsymbol{I} \end{pmatrix} \tag{4} $$

$$ {}^{\mathrm{W}}\boldsymbol{\Sigma}_{\mathrm{B}} = \boldsymbol{A}_{\mathrm{CB}}\, {}^{\mathrm{W}}\boldsymbol{\Sigma}_{\mathrm{C}} A_{\mathrm{CB}}^\top, \tag{5} $$

where ${}^{\mathrm{W}}\boldsymbol{d}$ is the lever arm vector from the camera to the robot base in the world frame, and $[\boldsymbol{v}]_\times$ is the skew-symmetric matrix of $\boldsymbol{v}$ such that $[\boldsymbol{v}]_\times \boldsymbol{w} = \boldsymbol{v} \times \boldsymbol{w}$ for any vector $\boldsymbol{w}$. Equation (4) is derived in `lever_arm_covariance.pdf`.